

2026-04-07 · Research

GLM-5.1: Towards Long-Horizon Tasks

 [Call it at Z.ai ↗](#)

 [Z.ai Coding Plan ↗](#)

 [GitHub ↗](#)

 [HuggingFace ↗](#)

GLM-5.1 is our next-generation flagship model for agentic engineering, with significantly stronger coding capabilities than its predecessor. It achieves state-of-the-art performance on SWE-Bench Pro and leads GLM-5 by a wide margin on NL2Repo (repo generation) and Terminal-Bench 2.0 (real-world terminal tasks).

- Software Engineering
- Repo Generation
- Agentic Coding
- Cybersecurity

Complex Software Engineering Tasks

SWE-Bench Pro

58.4	55.1	57.7	57.3	54.2

GLM-5.1 achieves state-of-the-art on complex software engineering tasks.

But the most meaningful leap goes beyond first-pass performance. Previous models—including GLM-5—tend to exhaust their repertoire early: they apply familiar techniques for quick initial gains, then plateau. Giving them more time doesn't help.

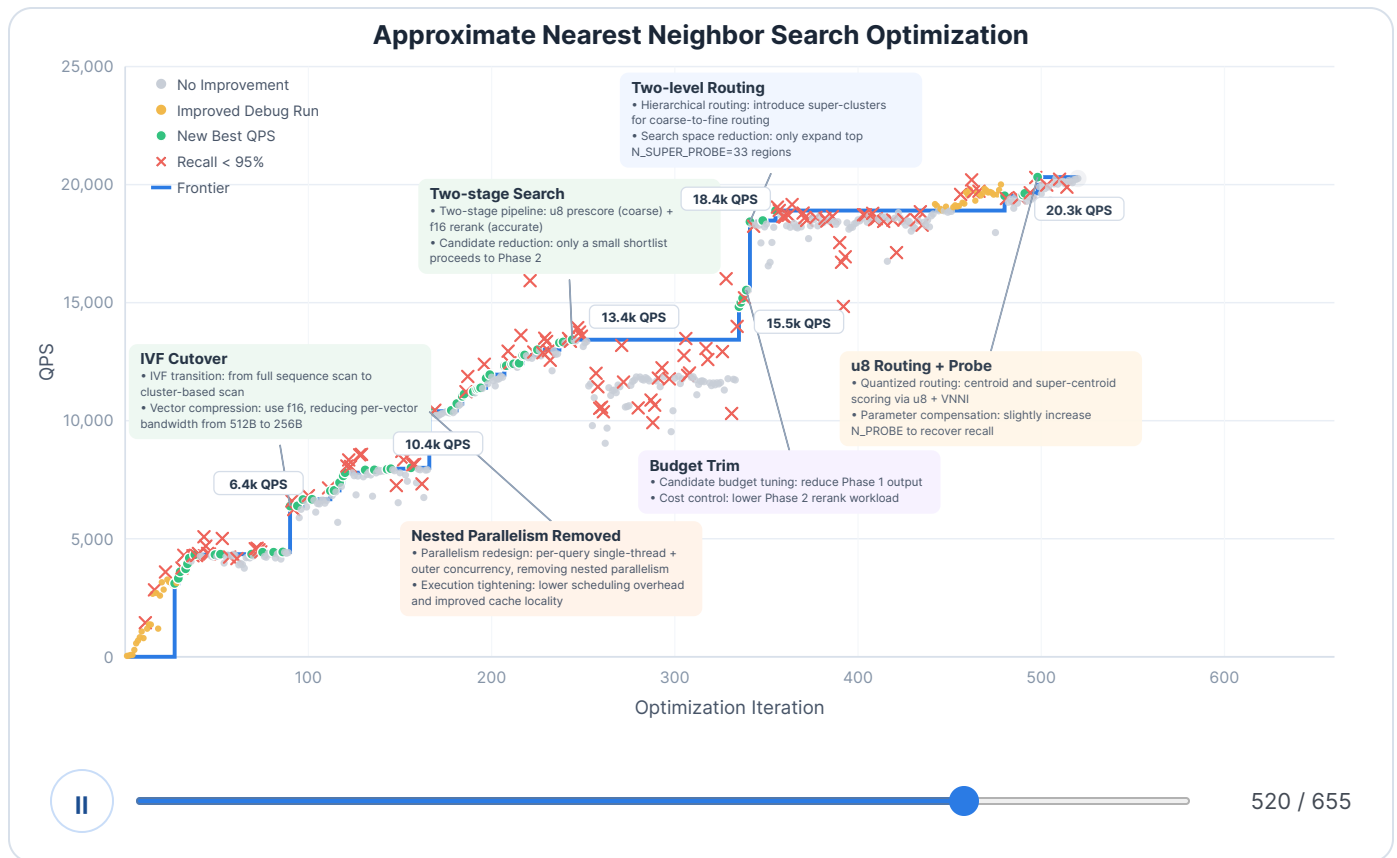
GLM-5.1, by contrast, is built to stay effective on agentic tasks over much longer horizons. We've found that the model handles ambiguous problems with better judgment and stays productive over longer sessions. It breaks complex problems down, runs experiments, reads results, and identifies blockers with real precision. By revisiting its reasoning and revising its strategy through repeated iteration, GLM-5.1 sustains optimization over hundreds of rounds and thousands of tool calls. The longer it runs, the better the result.

We demonstrate this across three tasks with progressively less structured feedback: a vector search optimization problem scored by a single numeric metric, a GPU kernel benchmark with per-problem speedup measurements, and an open-ended web application build where there is no metric at all—only the model's own judgment of what to improve next.

Scenario 1: Optimizing a Vector Database Over 600 Iterations

VectorDBBench is an open-source coding challenge that evaluates a model's ability to build a high-performance database to perform approximate nearest neighbor search. The model is given a Rust skeleton with HTTP API endpoints and empty implementation stubs, then uses tool-call-based agents to read and write files, compile, test, and profile—all within a 50-turn tool-call budget. The final result is benchmarked on the SIFT-1M dataset: models are ranked by QPS under the constraint that Recall $\geq 95\%$. The best result to date under this setting was 3,547 QPS, achieved by Claude Opus 4.6.

A natural question is whether this 50-turn budget is the bottleneck. We restructured the evaluation into an outer optimization loop with Claude Code framework: in each iteration, the model can use as many tool calls as needed to edit code, compile, test, and profile, then submit a new version to be benchmarked. The model decides autonomously when to submit and what to try next.



GLM-5.1 did not plateau after 50 or 100 submissions, but continued to find meaningful improvements over 600+ iterations with 6,000+ tool calls, ultimately reaching 21.5k QPS—roughly 6× the best result achieved in a single 50-turn session. The optimization trajectory shows a characteristic staircase pattern: periods of incremental tuning within a fixed strategy, punctuated by structural changes that shift the performance frontier.

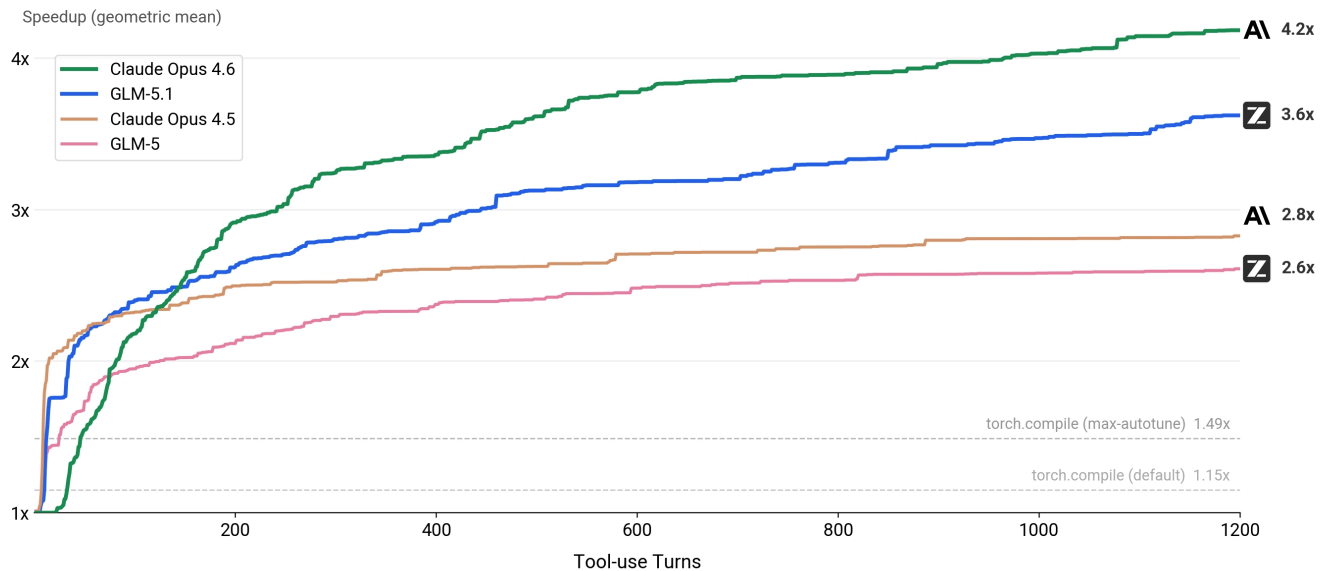
Two transitions illustrate the pattern. Around iteration 90, the model shifted from full-corpus scanning to IVF cluster probing with f16 vector compression, jumping to 6.4k QPS. Around iteration 240, it introduced a two-stage pipeline—u8 prescoring followed by f16 reranking—reaching 13.4k QPS. Six such structural transitions occurred over the full run, each initiated by the model after analyzing its own benchmark logs and identifying the current bottleneck. Red crosses in the chart mark iterations where Recall fell below 95%—these cluster around each major transition, as the model temporarily breaks the constraint while exploring a new direction, then adjusts to restore it.

Scenario 2: Optimizing Machine Learning Workload Over 1,000+ Turns

KernelBench evaluates whether a model can take a reference PyTorch implementation and produce a faster GPU kernel with identical outputs. The benchmark is organized into three levels

of increasing optimization scope and systems complexity: Level 1 covers single operators, Level 2 covers fused operator sequences, and Level 3 covers full-model, end-to-end optimization of complete architectures such as MobileNet, VGG, MiniGPT, and Mamba, for a total of 50 problems. For reference, torch.compile with default settings achieves 1.15× speedup on these problems; with max-autotune, 1.49×. We ran four models on Level 3, reporting the geometric mean speedup across all 50 problems as a function of tool-use turns.

KernelBench Level 3: Agentic ML Workload Optimization



The trajectories highlight differences in long-horizon optimization behavior. GLM-5 improves quickly at first but levels off relatively early. Claude Opus 4.5 continues a bit longer, but its gains also taper off in the later stages. GLM-5.1 pushes this frontier further, delivering 3.6× speedup and continuing to make progress well into the run. While its rate of improvement also slows over time, it sustains useful optimization for substantially longer than GLM-5. Claude Opus 4.6 remains the strongest model in this setting, finishing at 4.2× and still showing headroom at the end.

Scenario 3: Building a Linux Desktop Over 8 Hours

The previous two scenarios have explicit numeric objectives—QPS, speedup—that the model can benchmark against. Website generation is inherently more subjective: given a natural-language prompt, produce a working web application. There is no single metric to optimize; what counts as "good" depends on completeness, visual polish, and interaction quality.

We tested this with a deliberately ambitious prompt: build a Linux-style desktop environment as a web application. No starter code, no design mockups, no intermediate guidance. In a single run,

most models—including earlier versions of GLM—give up quickly: they produce a basic skeleton with a static taskbar and one or two placeholder windows, then declare the task complete. The model has no mechanism to step back and ask what's missing.

We wrapped GLM-5.1 in a simple harness that changes this: after each round of execution, the model reviews its own output, identifies what can be improved—missing features, rough styling, broken interactions—and continues. This loop ran for 8 hours, and the difference is substantial.



Early on, GLM-5.1 delivers a basic layout with a taskbar and simple window—similar to what a short session would produce. But it doesn't stop there. As it continues, the system steadily fills out: file browser, terminal, text editor, system monitor, calculator, games—each new addition integrated into a coherent UI rather than bolted on as an afterthought. Styling becomes more polished, interactions smoother, edge cases handled. By the end, the result is a complete, visually consistent desktop environment running in the browser—a concrete example of what becomes possible when the model is given the time and the capability to keep refining.

Across all three settings, the key variable is not runtime alone but whether additional runtime remains useful. GLM-5.1 extends that productive horizon meaningfully beyond GLM-5, while the remaining gap on tasks like KernelBench shows that long-horizon optimization is still an open frontier. There remain significant challenges: escaping local optima earlier when incremental tuning stops paying off, maintaining coherence over execution traces that span thousands of tool

calls, and—perhaps most importantly—developing reliable self-evaluation for tasks where there is no numeric metric to optimize against. GLM-5.1 is our first step in this direction, and we will continue to push on these fronts.

Benchmark	GLM-5.1	GLM-5	Qwen3.6-Plus	MiniMax M2.7	DeepSeek-V3.2	Kimi K2.5	Claude 3.7
REASONING							
HLE	31.0	30.5	28.8	28.0	25.1	31.5	31.5
HLE w/ Tools	52.3	50.4	50.6	-	40.8	51.8	51.8
AIME 2026	95.3	95.4	95.1	89.8	95.1	94.5	94.5
HMMT Nov. 2025	94.0	96.9	94.6	81.0	90.2	91.1	91.1
HMMT Feb. 2026	82.6	82.8	87.8	72.7	79.9	81.3	81.3
IMOAnswerBench	83.8	82.5	83.8	66.3	78.3	81.8	81.8
GPQA-Diamond	86.2	86.0	90.4	87.0	82.4	87.6	87.6
CODING							
SWE-Bench Pro	58.4	55.1	56.6	56.2	-	53.8	53.8
NL2Repo	42.7	35.9	37.9	39.8	-	32.0	32.0
Terminal-Bench 2.0 Terminus-2	63.5	56.2	61.6	-	39.3	50.8	50.8
Terminal-Bench 2.0 Best self-reported harness	66.5 (Claude Code)	56.2 (Claude Code)	-	57.0 (Claude Code)	46.4 (Claude Code)	-	-
CyberGym	68.7	48.3	-	-	17.3	41.3	41.3
AGENTIC							
BrowseComp	68.0	62.0	-	-	51.4	60.6	60.6
BrowseComp w/ Context Manage	79.3	75.9	-	-	67.6	74.9	74.9
τ^3 -Bench	70.6	69.2	70.7	67.6	69.2	66.0	66.0
MCP-Atlas Public Set	71.8	69.2	74.1	48.8	62.2	63.8	63.8
Tool-Decathlon	40.7	38.0	39.8	46.3	35.2	27.8	27.8

Benchmark	GLM-5.1	GLM-5	Qwen3.6-Plus	MiniMax M2.7	DeepSeek-V3.2	Kimi K2.5	Claude 3.7
Vending Bench 2	\$5,634.00	\$4,432.12	\$5,114.87	-	\$1,034.00	\$1,198.46	\$1,198.46

GLM-5.1 is released as open source under the MIT License. GLM-5.1 is also available on the developer platforms [api.z.ai](#) and [BigModel.cn](#), and is compatible with Claude Code and OpenClaw.

Getting started with GLM-5.1

Use GLM-5.1 with GLM Coding Plan

Try **GLM-5.1** in your favorite coding agents—**Claude Code**, **OpenCode**, **Kilo Code**, **Roo Code**, **Cline**, **Droid**, and more. <https://docs.z.ai/devpack/overview>

For GLM Coding Plan subscribers: We’re rolling out GLM-5.1 to all Coding Plan users. You can enable GLM-5.1 now by updating the model name to "GLM-5.1" (e.g. in `~/.claude/settings.json` for Claude Code). As our most capable model, GLM-5.1 consumes quota at 3× during peak hours and 2× during off-peak hours. As a limited-time promotion through the end of April, off-peak usage is billed at 1×. (Peak hours are 14:00–18:00 UTC+8 (Beijing Time) daily)

Prefer a GUI? We offer **Z Code** —one interface, multiple agents, working together. Develop on remote machines over SSH, or kick off tasks from your phone and check back later.

Start building now: <https://z.ai/subscribe>

Chat with GLM-5.1 on Z.ai

GLM-5.1 will be available on [Z.ai](#) in the coming days.

Serve GLM-5.1 Locally

The model weights of GLM-5.1 are publicly available on [HuggingFace](#) and [ModelScope](#). For local deployment, GLM-5.1 supports inference frameworks including vLLM and SGLang. Comprehensive deployment instructions are available at the official GitHub repository.

Footnote

- **Humanity's Last Exam (HLE) & other reasoning tasks:** We evaluate with a maximum generation length of 163,840 tokens (temperature=1.0, top_p=0.95, max_new_tokens=163840). By default, we report the text-only subset; results marked with * are from the full set. We use GPT-5.2 (medium) as the judge model. For HLE-with-tools, we use a maximum context length of 202,752 tokens.
- **SWE-Bench Pro:** We run the SWE-Bench Pro suite with OpenHands using a tailored instruction prompt. Settings: temperature=1, top_p=0.95, max_new_tokens=32768, with a 200K context window.
- **NL2Repo:** We evaluated NL2Repo with temperature=1.0, top_p=1.0, and max_new_tokens=32768 under 200k context. To prevent hacking, we use rule-based pre-detection for malicious commands (e.g., unauthorized pip or curl operations), followed by a model-based judgement. Malicious action is immediately intercepted.
- **BrowserComp:** Without context management, we retain details from the most recent 5 turns. With context management, we use the same discard-all strategy as GLM-5 and DeepSeek-v3.2.
- **Terminal-Bench 2.0 (Terminus 2):** We evaluate with the Terminus framework using timeout=3h, temperature=1.0, top_p=1.0, max_new_tokens=8192, with a 200K context window. Resource limits are capped at 16 CPUs and 32 GB RAM.
- **Terminal-Bench 2.0 (Claude Code):** We evaluate in Claude Code 2.1.14 (think mode) with temperature=1.0, top_p=0.95, max_new_tokens=131072. We remove wall-clock time limits, while preserving per-task CPU and memory constraints. We fix environment issues introduced by Claude Code and also report results on a verified Terminal-Bench 2.0 dataset that resolves ambiguous instructions (see: <https://huggingface.co/datasets/zai-org/terminal-bench-2-verified>). Scores are averaged over 5 runs.
- **CyberGym:** We evaluate in Claude Code 2.1.56 (think mode, no web tools) with (temperature=1.0, top_p=1.0, max_new_tokens=32000) and a 250-minute timeout per task. Results are single-run Pass@1 over 1,507 tasks.
- **MCP-Atlas:** All models were evaluated in think mode on the 500-task public subset with a 10-minute timeout per task. We use Gemini-3.0-Pro as the judge model for evaluation.
- **τ^3 -bench:** An additional prompt was added to the user simulator across all domains to avoid failure modes caused by users ending the interaction prematurely. Banking domain uses terminal-based agentic search retrieval (terminal_use). User simulator: GPT-5.2 (reasoning_effort: low), 4 trials.
- **Vending Bench 2:** Runs are conducted independently by [Andon Labs](#).
- **KernelBench Level 3:** Each of the 50 problems runs in an isolated Docker container with one H100 GPU, limited to 1200 tool-use turns. Correctness (atol=rtol=1e-4) and performance are evaluated against the PyTorch eager baseline in separate CUDA contexts. All solutions are

independently audited for benchmark exploitation by Claude Opus 4.6 (max effort) and GPT-5.4 (xhigh): each audit verifies the optimization does not exploit benchmark-specific behavior, works with arbitrary new inputs, and keeps all computation on the default CUDA stream. The lower speedup across audits is used, with a 50× hard cap to limit the influence of outliers.



Legal

[Privacy Policy](#)

[Terms of Service](#)

© 2026 [Z.ai](#) Inc.

